

UNIT - III

SYLLABUS

Distributed deadlock detection : Introduction, deadlock handling strategies in distributed system, centralized and distributed deadlock detection algorithms.

Agreement protocols : Introduction, system model, classification of agreement problems, solutions to the Byzantine agreement problem.

INTRODUCTION

In this unit we will study about distributed deadlock detection and agreement protocols. A distributed system is a network of sites that exchange information with each other by message passing. We will see deadlock handling strategies in distributed systems then we will study agreement protocols for distributed systems under processor failures.

In agreement problems, nonfaulty processors in a distributed system should be able to reach a common agreement, even if certain components in the system are faulty. The agreement is achieved through an agreement protocol that involves several rounds of message exchange among the processors.

DEADLOCK

Q.1. What is deadlock?

Ans. Deadlock :

- A deadlock is a condition in a system where a set of processes (or threads) have requests for resources that can never be satisfied. Essentially, a process cannot proceed because it needs to obtain a resource held by another process but it itself is holding a resource that the other process needs.
- More formally, Coffman defined four conditions have to be met for a deadlock to occur in a system :

(1) Mutual exclusion :

A resource can be held by at most one process.

(2) Hold and wait :

Processes that already hold resources can wait for another resource.

(3) Non-preemption :

A resource, once granted, cannot be taken away from a process.

(4) Circular wait :

Two or more processes are waiting for resources held by one of the other processes.

- A directed graph model is used to record the resource allocation state of a system. This state consists of n processes, $P_1 \dots P_m$ and m resource $R_1 \dots R_m$.
- In such a graph,
 $P_i \rightarrow R_j$ means that resource R_j is allocated to process P_i .
 $P_i \leftarrow R_j$ means that resource R_j is requested by process P_i .

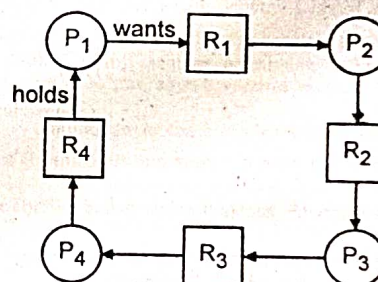


Fig. (a) Deadlock

- Deadlock is present when the graph has cycles. An example is shown in Fig. (a) such a graph is called as a Wait-For Graph (WFG).

Deadlock in distributed system :

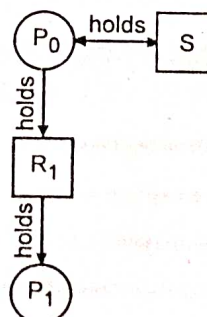


Fig. (b) Resource graph on S

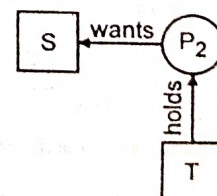


Fig. (c) Resource graph on T

- The same conditions for deadlock in uniprocessors apply to distributed systems. Unfortunately, as in many other aspects of distributed systems, they are harder to detect, avoid and prevent.
- Four strategies can be used to handle deadlock :
 - (1) **Ignorance :**
Ignore the problem; assume that a deadlock will never occur. This is a surprisingly common approach.
 - (2) **Detection :**
Let a deadlock occur, detect it, and then deal with it by aborting and later restarting a process that causes deadlock.
 - (3) **Prevention :**
Make a deadlock impossible by granting requests so that one of the necessary conditions for deadlock does not hold.
 - (4) **Avoidance :**
Choose resource allocation carefully so that deadlock will not occur. Resource requests can be honored as long as the system remains in a safe (non-deadlock) state after resources are allocated.
- The last of these, deadlock avoidance through resource allocation is difficult and requires the ability to predict precisely the resources that will be needed and the times that they will be needed. This is difficult and not practical in real systems.
- The first of these is trivially simple but, of course, ineffective for actually doing anything about deadlock conditions.

DISTRIBUTED DEADLOCK DETECTION

Q.2. What is distributed deadlock detection?

Ans. Distributed deadlock detection :

- A distributed system is a network of sites that exchange information with each other by message passing.
- A site consists of computing and storage facilities and interface to local user and communication network.
- In distributed system, a process can request and release resources (local or remote) in any order, which may not be known a priori and process can request some resources while holding others.
- If the sequence of allocation of resources to processes is not controlled in such environments, deadlock can occur.
- A deadlock occurs when processes holding some resources request access to resources held by other processes in the same set.

- The simplest illustration of a deadlock consists of two processes, each holding a different resource in exclusive mode and each requesting an access to resources held by other processes.
- Unless the deadlock is resolved, all the processes involved are blocked indefinitely. Therefore, a deadlock requires the attention of a process outside those involved in the deadlock for its detection and resolution.
- A deadlock is resolved by aborting one or more processes involved in the deadlock and granting the released resources to other processes involved in the deadlock.
- A process is aborted by withdrawing all its resource requests, restoring its state to an appropriate previous state, relinquishing all the resources it acquired after that state, to their original states. In the simplest case, a process is aborted by starting it afresh and relinquishing all the resources it held.

Q.3. Explain resource and communication deadlock model in detail.

Ans.

- There are two types of deadlock models, resource model and communication model.
- (1) **Resource deadlock model :**
 - In resource model, processes make requests to acquire some resources, such as data objects in distributed database. Processes can simultaneously wait for several resources and cannot proceed until it acquired all the locks of those resources.
 - A set of processes is resource-deadlocked if all processes in the set are waiting for some other processes in the set to release the lock of some resources.
 - The system model divided the communication message into two categories; computation message and control message.
 - A computation message is any message that is sent because of the execution of an application process. It can either be REQUEST, REPLY, or CANCEL message. REQUEST message is sent to a resource controller when a process wants to access to that resource. REPLY message is sent to a requesting process when the resource controller has determined that the resource is ready to be access by that process.

- A CANCEL message is sent to a requested resource controller when the process decides to cancel the request made to access that resource earlier. A resource requested by a process is said to be in the dependent set of the process. A control message is any message that is sent because of the execution of the deadlock detection algorithm.

- Resources are reusable.
- Resources are not duplicated.
- No two process can access a resource at the same time.

- A process P_j is said to be dependent on another process P_k if there exists a sequence of processes $P_j, P_{i(1)}, P_{i(2)}, P_{i(3)}, \dots, P_k$, where each process in the sequence is idle and each process (except the first) in the sequence holds a resource for which the previous process in the sequence is waiting. P_j is said to be locally dependent on P_k if all the processes in the sequence is located in the same site.

(2) Communication deadlock model :

- In communication model, the resources, processes are acquiring messages. Blocked processes are those that are waiting for several arrivals of messages at the same time. A process can be unblocked when a subset of those messages it is waiting is arrived.
- A nonempty set of processes is communication-deadlocked if all processes in the set are permanently idle. A process is permanently idle if it never receives a message from any process in its dependent set. However, permanent idle cannot be detected by timeout since we cannot say that a process A and B are deadlock on each other if B eventually send a message to A as the time B sends a message is unknown.
- The Chandy, Misra and Haas defined a set of process S is deadlocked if:
 - All processes in S are idle.
 - The dependent set of every process in S is a subset of S.
 - There are no messages in transit between processes S.
- As all processes are idle and dependent on some processes in the same set, there can be no message transits within the set. Without message transits and all processes are dependent on some processes in the same set, all processes are idle.

Q.4. What is Graph-Theoretic model? Explain about Wait-For Graph (WFG).

Ans. Graph-theoretic model :

- The state of process-resource interaction in distributed systems can be modeled by a bi-partite directed graph called a resource allocation graph.
- The nodes of this graph are processes and resources of a system, and the edges of the graph depict assignment or pending requests.
- A pending request is represented by a request edge directed from the node of a requesting process to the node of the requested resource.
- A resource assignment is represented by an assignment edge directed from the node of an assigned resource to the node of the assigned process.
- A system is deadlocked if its resource allocation graph contains a directed cycle or a knot.

Wait-For Graph :

- In distributed systems, the system state can be modeled or represented by a directed graph, called a Wait-For Graph (WFG). In a WFG, nodes are processes and there is a directed edge from node P_1 to node P_2 , if P_1 is blocked and is waiting for P_2 to release some resource.
- A system is deadlocked if and only if there is a directed cycle or knot in the WFG.
- In Distributed Database System (DDBS), users access the data objects of the database by executing transactions. A transaction can be viewed as a process that performs a sequence of reads and writes on the data objects.
- The data objects of a database can be viewed as resources that are acquired (through locking) and released (through unlocking) by transactions.
- A wait-for graph is also referred to as a transaction-wait for graph (TWF graph). In a TWF graph, nodes are transactions and there is a directed edge from node T_1 to node T_2 if T_1 is blocked and is waiting for T_2 , to release some resource. A system is deadlocked if and only if there is a directed cycle or a knot in its TWF graph.

DEADLOCK HANDLING STRATEGIES IN DISTRIBUTED SYSTEM

Q.5. Discuss deadlock handling strategies in distributed system.

OR How to prevent deadlock in distributed system?

OR Write short note on deadlock avoidance.

Ans. There are three strategies to handle deadlocks :

(1) Deadlock prevention.

(2) Deadlock avoidance.

(3) Deadlock detection.

- Deadlock handling is complicated to implement in distributed systems because no one site has accurate knowledge of the current state of the system and because every intersite communication involves a finite and unpredictable delay.

(1) Deadlock prevention :

- Deadlock prevention is commonly achieved by either having a process acquire all the needed resources simultaneously before it begins execution or by pre-empting a process that holds the needed resource.
- In the formal method, a process requests (or release) a remote resource by sending a request message (or release message) to the site where the resource is located.
- This method has number of drawbacks :
 - (i) It is inefficient as it decreases the system concurrency.
 - (ii) A set of processes can become deadlocked in the resource acquiring phase.
- For example, suppose process P_1 at site S_1 and process P_2 , at site S_2 , simultaneously request two resources R_3 , and R_4 , located at sites S_3 , and S_4 , respectively. It may happen that S_3 , grants R_3 , to P_1 and S_4 , grants R_4 , to P_2 , resulting in a deadlock.
- This problem can be handled by forcing processes to acquire needed resources one by one, but this approach is highly inefficient and impractical.

(2) Deadlock avoidance :

- In deadlock avoidance approach to distributed systems, a resource is granted to a process if the resulting global system state is safe. Global state includes all the processes and resources of the distributed system.

- Because of the following problems, deadlock avoidance can be seen as impractical in distributed systems:
 - (i) Every site has to maintain information on the global state of the system, which translates into huge storage requirements and extensive communication costs (as every change in the global state has to be communicated to every site).
 - (ii) The process of checking for a safe global state must be mutually exclusive, because if several sites concurrently perform checks for a safe state (each site for a different resource request), they may all find the state safe but the net global state may not be safe. These restrictions will severely limit the concurrency and throughput of the system.
 - (iii) Due to large number of processes and resources, it will be computationally expensive to check for a safe state.
- (3) Deadlock detection :**
 - Deadlock detection requires an examination of the status of process-resource interactions for the presence of cyclical wait.
 - Deadlock detection in distributed systems has two favourable conditions:
 - (i) Once a cycle is formed in the WFG (Wait-for-Graphs), it persists until it is detected and broken.
 - (ii) Cycle detection can proceed concurrently with the normal activities of a system (and therefore it does not have a negative effect on system throughput).
 - Because of this, deadlock handling in distributed system is highly focused on deadlock detection methods.

Q.6. What are the issues in deadlock detection and resolution?

Ans.

- Deadlock handling using the approach of deadlock detection entails addressing two basic issues:
 - (1) Detection of existing deadlocks.
 - (2) Resolution of detected deadlocks.
- (1) Detection :**
 - Detection of deadlocks involves addressing two issues:
 - (i) Maintenance of the WFG.
 - (ii) Searching of the WFG for the presence of cycles (or knots).

CONTROL ORGANIZATION FOR DISTRIBUTED DEADLOCK DETECTION

Q.7. Explain control organization strategies for distributed deadlock detection.

Ans. The control organization strategies for distributed deadlock detection are as follows :

(1) Centralized control :

- In centralized deadlock detection algorithms, a designated site, often called the control site, has the responsibility of constructing the global WFG and searching it for cycles.
- The control site may maintain the global WFG all the time, or it may build it whenever deadlock detection is to be carried out by soliciting the local WFG from every site.
- Centralized algorithms are conceptually simple and are easy to implement.
- Deadlock resolution is simple in these algorithms, the control site has the complete information about the deadlock cycle, and it can optimally resolve the deadlock.
- However, because control is centralized at a single site, centralized deadlock detection algorithms have a single point of failure.
- Communication links near the control site are likely to be congested because the control site receives WFG information from all the other sites.
- Also, the message traffic generated by deadlock detection activity is independent of the rate of deadlock formation and the structure of deadlock cycles.

(2) Distributed control :

- In distributed deadlock detection algorithms, the responsibility of detecting a global deadlock is shared equally among the sites.
- The global state graph is spread over many sites, and several sites participate in the detection of a global cycle.
- Unlike centralized algorithms, distributed algorithms are not vulnerable to a single point of failure, and no site is swamped with deadlock detection activity.
- Deadlock detection is initiated only if a waiting process is suspected to be part of a deadlock cycle.

- Because in distributed systems a cycle may involve several sites, the search for cycles greatly depends on how the system state graph is represented across the system.
- Depending upon the manner in which state graph is maintained and the search for the cycle is carried out, the three types of algorithms for deadlock detection in distributed systems are centralized, distributed, and hierarchical algorithms.
- A deadlock detection algorithm must satisfy the following two conditions also known as correctness criteria.

(a) Progress (No undetected deadlocks) :

- The algorithm must detect all existing deadlocks in finite time.
- Once a deadlock has occurred, the deadlock detection activity should continuously progress until the deadlock is detected.
- In other words, after all wait-for dependencies for a deadlock have formed, the algorithm should not wait for any more events to occur to detect the deadlock.

(b) Safety (No false deadlocks) :

- The algorithm should not report deadlocks which do not exist (called phantom or false deadlocks).
- In distributed systems where there is no global memory and communication occurs solely by messages, it is difficult to design a correct deadlock detection algorithm because site may obtain out of date and inconsistent graph of the system.
- As a result, site may detect a cycle that doesn't exist, but whose different segments were existing in the system at different times.

Resolution :

- Deadlock resolution involves breaking existing wait-for dependencies in the system WFG to resolve the deadlock.
- It involves rolling back one or more deadlocked processes that are deadlocked and assigning their resources to blocked processes in the deadlock so that they can resume execution.
- Note that several deadlock detection algorithms propagate information regarding wait-for dependencies along the edges of the wait-for graph.
- Therefore, when a wait-for dependency is broken, the information is not cleaned appropriately in a timely manner, it may result in detection of phantom deadlock.

- But deadlock resolution is often cumbersome in distributed deadlock detection algorithms because several sites may detect the same deadlock and may not be aware of other sites and/or processes involved in the deadlock.
- Distributed algorithms are difficult to design because sites may collectively report the existence of a global cycle after seeing its segments at different instants (though all the segments never existed simultaneously) due to the system's lack of globally shared memory.
- Also, proof of correctness is difficult for these algorithms.

(3) Hierarchical control :

- In hierarchical deadlock detection algorithms, sites are arranged in a hierarchical fashion, and a site detects deadlocks involving only its descendant sites. Hierarchical algorithms exploit access patterns local to a cluster of sites to efficiently detect deadlocks.
- They tend to get the best of both the centralized and the distributed control organization in that there is no single point of failure (as in centralized control) and a site is not bogged down by deadlock detection activities with which it is not concerned (as sometimes happens in distributed control).
- However, hierarchical deadlock detection algorithms require special care while arranging the sites in a hierarchy. For efficiency, most deadlocks should be localized to as few clusters as possible, the objective of hierarchical control is defeated if most deadlocks span several clusters.

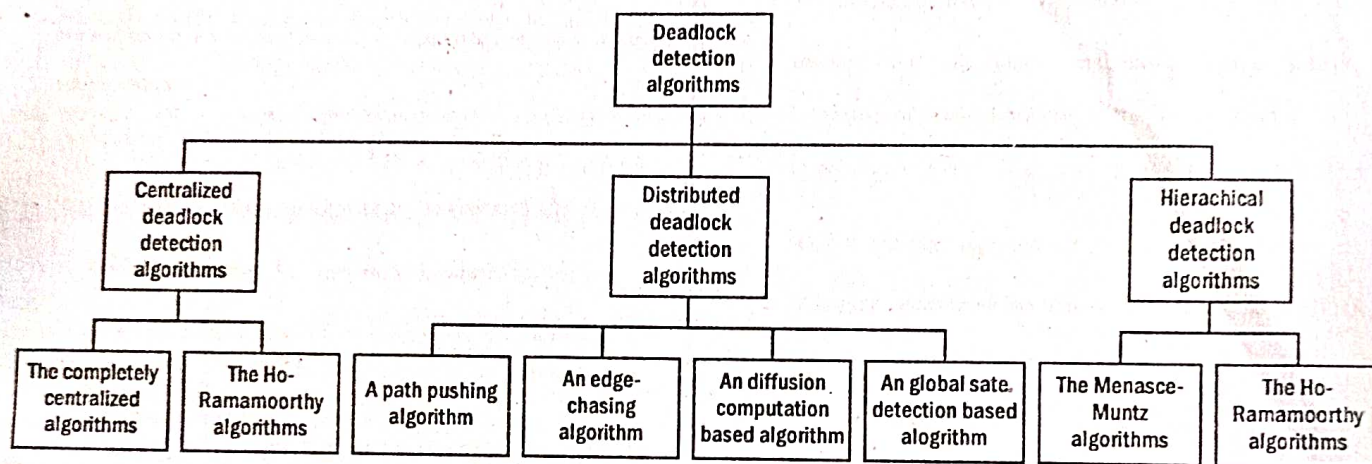


Fig. Deadlock Detection algorithm in Distributed OS

CENTRALIZED DEADLOCK DETECTION ALGORITHMS

Q.8. Explain completely centralized algorithm in detail.

Ans. The completely centralized algorithm :

- The completely centralized algorithm is the simplest centralized deadlock detection algorithm, wherein a designated site called the control site, maintains the WFG of the entire system and checks it for the existence of deadlock cycles.
- All sites request and release resources (even local resources) by sending request resource and release resource messages to the control site respectively.
- When the control site receives a request resource or a release resource message, it correspondingly updates its WFG.

- The control checks the WFG for deadlocks whenever a request edge is added to the WFG.
- This algorithm is conceptually simple and easy to implement, but is also highly inefficient because all resources acquisition and release request must go through the control site, even when the resource is local. This results in large delays in responding to user requests, large communication overhead, and the congestion of communication links near the control site. Also the reliability is poor because if the control site fails, entire system comes to halt.
- Several problems of this algorithm (such as large response time and the congestion of the communication links near the control site) can be mitigated by having each site maintain its resource status (WFG) locally and by having each site send its resource status to a designated site periodically for construction of the global WFG and detection of deadlocks.

- However due to inherent communication delays and the lack of perfectly synchronized clocks, the designated site may get an inconsistent view of the system and detect false deadlocks.
- For example, suppose two resources R_1 and R_2 are stored at sites S_1 and S_2 , respectively. Suppose the following two transactions T_1 and T_2 are started almost simultaneously at sites S_3 and S_4 , respectively :

T_1	T_2
Lock R_1	Lock R_1
Unlock R_1	Unlock R_1
Lock R_2	Lock R_2
Unlock R_2	Unlock R_2

- Suppose the Lock (R_1) request of T_1 arrives at S_1 and locks R_1 , followed by the Lock (R_1) request of T_2 , which waits at S_1 . At this point S_1 reports its status, $T_2 \rightarrow T_1$, to the designated site.
- Therefore, T_1 unlocks R_1 , T_2 locks R_1 , T_1 makes a lock (R_2) request to S_2 , T_2 unlocks R_1 and makes a lock (R_2) request to S_2 .
- Suppose the Lock (R_2) request of T_2 arrives at S_2 and locks R_2 , followed by the Lock (R_2) request of T_1 , which waits at S_2 . At this point S_2 reports its status, $T_1 \rightarrow T_2$, to the designated site, which after constructing the global state graph, reports a false deadlock $T_1 \rightarrow T_2 \rightarrow T_1$.

Q.9. Explain Ho-Ramamoorthy one phase algorithm.

Ans. The Ho-Ramamoorthy one-phase algorithm :

- The one-phase algorithm requires only one phase of status reports from sites, but each site maintains two status tables: a resource status table and a process status table.
- The resource status table keeps track of the transactions that have locked or are waiting for resources stored at that site.
- The process status table keeps track of the resources locked by or waited for by all the transactions at that site.

- Periodically, a designated site requests both tables from all other sites, constructs a state graph using only the transactions for which the resource table matches the process table, and searches it for cycles.
- If no cycle is found, then the system is not deadlocked; otherwise, a deadlock has been detected.
- The algorithm does not detect false deadlocks because it eliminates inconsistency in state information by using only the information common to both tables.
- For example, if the resource table at S_1 indicates that resource R_1 is being waited for by a process P_2 (i.e., $R_1 \rightarrow P_2$) and the process table at S_2 indicates that process P_2 is waiting for resource R_2 (i.e., $P_2 \rightarrow R_2$), then edge $P_2 \rightarrow R_1$ in the constructed state graph correctly reflects the system state. If either of these entries is missing from the resource or process table, then a request message or release message from S_2 to S_1 is in transit and $P_2 \rightarrow R_1$ cannot be ascertained.
- The one-phase algorithm is faster and requires fewer messages than the two-phase algorithm.
- But it requires more storage because every site maintains and exchanges two status tables.

Q.10. Explain Ho-Ramamoorthy two phase algorithm.

Ans. The Ho-Ramamoorthy two-phase algorithm :

- In this algorithm every site maintains a status table, which contains the status of all processes initiated at that site.
- The status of a process includes all resources locked and all resources being waited upon.
- Periodically, a designated site requests the status table from all sites, constructs a state graph (WPG) from the received information and searches it for cycles.
- If there is no cycle, then the system is not deadlocked. Otherwise, the designated site again requests status tables from all the sites and again constructs a state graph, using only the transactions common to both reports.

- If the same cycle is detected again, the system is declared deadlocked.
- Ho and Ramamoorthy claimed that by selecting the transactions common to two consecutive reports, the designated site gets a consistent view of the system (a view that correctly reflects the state of the system)
- Because if a deadlock exists, the same wait-for condition must exist in both reports.
- However, this claim is incorrect – a cycle in the wait-for conditions of the transactions common to two consecutive reports does not imply a deadlock – and the two-phase algorithm can report false deadlocks.
- By getting two consecutive reports, the designated site reduces but does not eliminate the probability of getting an inconsistent view, but does not eliminate such a possibility.

DISTRIBUTED DEADLOCK DETECTION ALGORITHMS

Q.11. Write a short note on distributed deadlock detection algorithm.

Ans. Distributed deadlock detection algorithm :

- In distributed deadlock detection algorithms, all sites collectively cooperate to detect a cycle in the state graph that is likely to be distributed over several sites of the system.
- A distributed deadlock detection algorithm can be initiated whenever a process is forced to wait, and it can be initiated either by the local site of the process or by the site where the process wait.
- Distributed deadlock detection algorithms can be divided into four classes:
 - (1) Path-pushing
 - (2) Edge-chasing
 - (3) Diffusion computation
 - (4) Global state detection.
- In path-pushing algorithms, the wait-for dependency information of the global WFG is disseminated in the form of paths (i.e. a sequence of wait-for dependency edges). Classic examples of such algorithms are Menasce-Muntz and Obermarck algorithms.
- In edge-chasing algorithms, special messages called probes are circulated along the edges of the WFG to detect a cycle. Examples of these algorithms include Chandy and Sinha-Natarajan algorithms.

- Diffusion computation type algorithms make use of echo algorithms to detect deadlocks. To detect a deadlock, a process sends out query messages along all the outgoing edges in the WFG. These queries are successively propagated through the edges of the WFG. (As the name implies, these queries are diffused through the WFG). Queries are discarded by a running process and are echoed back by blocked processes in the following way : When a blocked process receives first query message for a particular deadlock detection initiation, it does not send a reply message until it has received a reply message for every query it sent (to its successors in the WFG). For all subsequent queries for this deadlock detection initiation, it immediately sends back a reply message. The initiator of a deadlock detection detects a deadlock when it receives a reply for every query it sent. Some examples of these types of deadlock detection algorithms are Chandy-Misra-Haas algorithm for the OR request model and Chandy-Herman algorithm.

- Global state detection based deadlock detection algorithms exploit the following facts :
 - (i) A consistent snapshot of a distributed system can be obtained without freezing the underlying computation.
 - (ii) A consistent snapshot may not represent the system state at any moment in time, but if a stable property holds in the system before the snapshot collection is initiated, this property will still hold in the snapshot.
- Therefore, distributed deadlocks can be detected by taking a snapshot of the system and examining it for the condition of a deadlock.

Q.12. Explain path-pushing algorithm of distributed deadlock detection.

Ans. Path-pushing algorithm :

- In path-pushing algorithms, distributed deadlocks are detected by maintaining an explicit global WFG.
- The basic idea is to build a global WFG for each site of the distributed system.
- In this class of algorithms, at each site whenever deadlock computation is performed, it sends its local WFG to all the neighbouring sites.

- After the local data structure of each site is updated, this updated WFG is then passed along to other sites, and the procedure is repeated until some site has a sufficiently complete picture of the global state to announce deadlock or to establish that no deadlocks are present.
- This feature of sending around the paths of global WFG has led to the term path-pushing algorithms.
- Classic examples of such algorithms are Menasce-Muntz and Obermarck algorithms.

Q.13. Explain Menasce-Muntz algorithm.

Ans. Menasce-Muntz algorithm :

- The deadlock detection algorithm of Menasce and Muntz propagates only the two end points of a directed path (called a blocking pair), rather than the whole path, to detect deadlocks.
- The blockingset(T) of a transaction T is the set of all non-blocked transactions that can be reached from T by following all directed paths in the TWF graph.
- This is the set of transactions responsible for blocking the transaction T .
- When a transaction T gets blocked, then for each transaction T_i in the blockingset(T), the algorithm sends the blocking pair (T, T_i) to the home sites of T and T_i . In other words, information about the condensed TWF graph is sent along the paths of the global TWF graph.
- Fig. below illustrates the algorithm for a deadlock involving three transactions T_1, T_2 and T_3 .
- Initially T_1 is blocked by T_2 and T_2 by T_3 , and the home sites of T_1 and T_2 have the knowledge of the TWF graph $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_3$, respectively.
- Now, when T_3 makes a request and is blocked by T_1 , the blocking pair (T_3, T_2) is sent to the home site of T_2 .
- This causes an edge from T_3 to T_2 to be added in the TWF of the home site of T_2 , resulting in a cycle $T_3 \rightarrow T_2 \rightarrow T_3$ and detection of a deadlock at the home site of T_2 .

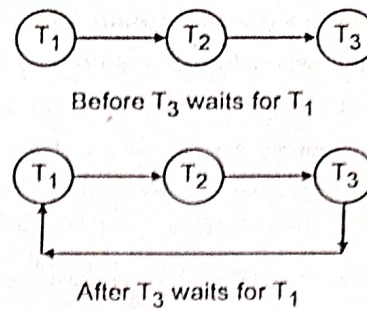


Fig. Example of Menasce-Muntz algorithm

Q.14. Explain Obermarck's algorithm in detail.

Ans. Obermarck's algorithm :

- Obermarck's algorithm in deadlock detection is a path-pushing algorithm.
- Path-pushing algorithms are those in which information of the global WFG is distributed in the form of paths. It was developed for the disturbed database system R^* of the IBM Corporation. In a distributed database system, the computation is done in a set of participating sites.
- A transaction is an abstraction for the application processing performed to take the database from one consistent state to another consistent state in a way that it can be viewed as atomic.
- Transactions involve agents in different sites. Therefore, given a site, there can be multiple agents in the site working on different or the same transaction. When the first transaction is waiting for the second transaction and the second transaction is waiting for the first transaction at the same time, the system is said to be deadlocked.
- The term Transaction Wait-For-Graph (TWFG) is used to stress the difference between transaction model and communications model. However, the mechanism of the graph is roughly the same as WFG.
- Each transaction is represented by a globally unique identifier. Since each site consists of a number of agents that belongs to different transaction, the global TWFG is split into smaller parts when viewed by each site.
- In a given site S , the algorithm uses a non-existing virtual agent EXTERNAL ("EX") to denote external sites that are not S . Only one $T_1 \rightarrow T_2$ edge is created in the TWFG no matter how many times transaction T_1 is waiting for transaction T_2 .

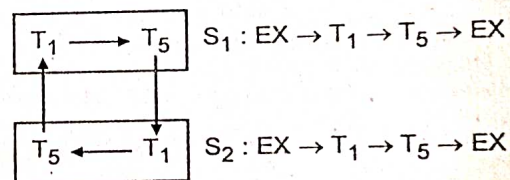
- The algorithm at each site builds and analyzes a directed TWFG, where the nodes represent the transaction agents and the edges denote the wait-for relationships. When an agent is dependent (waiting for) some agents in external site, it is denoted by an edge to "EX". Similarly, when an agent is determined that an agent in external sites is waiting for itself, it is represented by an edge from "EX" to that agent.
- For example, in a TWFG $EX \rightarrow T_1 \rightarrow T_2 \rightarrow EX$ denoted a potential global deadlock that involves some external sites, where there is an agent in external sites is waiting for T_1 and T_2 is waiting for some agent in external sites.
- The deadlock detectors are roughly synchronized when they exchange control messages. The control messages are in the form of strings. Strings can be thought as partial TWFG that a site S_1 sends to site S_2 when some agents in S_1 are waiting for some agents in S_2 . For the previous example of TWFG, the site will send the string "EX", TRANS1, TRANS2 to the site where T_2 is waiting for.
- Elementary cycles are cycles that do not involve EXTERNAL. These cycles are detected by each site in step 5 of the algorithm. When a cycle is detected, the site will choose a victim in the cycle such that it will be removed.

Algorithm :

For each site S ,

- (1) Build a wait-for-graph using the transaction-to-transaction wait-for relationships.
- (2) Receive any strings of nodes transmitted from other sites and add them into the wait-for-graph.
- (a) For each transaction identified in the string, create a node of the TWFG if none exists at this site.
- (b) For each transaction identified in the string, starting with the first, create an edge to the node representing the next transaction in the string.
- (3) Create wait-for edges from EXTERNAL to each node representing a transaction's agent that is expected to received on a communication link.
- (4) Create wait-for edges to EXTERNAL from each node representing a transaction's agent that is expected to send on a communication link.

- (5) Analyze the resulting graph, listing all elementary cycles.
- (6) Select a victim to break each cycle that does not contain the node External. As each victim is chosen for a given cycle, remove all cycles that include the victim.
- (a) Site must remember the transaction identifier of the victim such that it can discard strings received involves the victim.
- (b) If the victim transaction has an agent at this site, then the fact that the transaction was chosen as a victim must be transmitted to each site known to contain an agent of the victim transaction. Otherwise, the site has to transmit the fact to each site that sends a string containing the victim's identifier to S .
- (7) Examine each remaining cycle that contains the node External. If the transaction identifier of the node External is waiting for is greater than the node that waits for External, then
 - (a) Transform the cycle into a string, which starts with "EX" and terminates with a node identifier that identify the node waiting for External in the site.
 - (b) Send the string to each site which the terminating node in the string is waiting for.

**False deadlocks :**

- False deadlocks are deadlocks that are detected but do not really exist in the system. The algorithm assumed that the state of the system is static and strings are propagated to other sites to determine a deadlock. However, it is obvious that the state of a distributed system can be rather dynamic. A local TWFG may not be a real picture of the global state.
- As shown in the above figure, suppose a deadlock detection algorithm determined that $T_1 \rightarrow T_5$ exists in site S_1 . At the time the detection algorithm is querying the T_2, T_3 and T_4, T_5 can release its locks and T_1 waits is finished. However, the detection algorithm will not know about that and sends the string to other site S_2 . When

S_2 with TWFG $T_1 \rightarrow T_3$ received such a string, it will determine that a deadlock is existing $T_1 \rightarrow T_3 \rightarrow T_1$, while it is not necessarily true.

- Based on the fact real deadlocks will persist until broken, when a deadlock cycle is detected, validation of the transaction wait-for relationships in the cycle can be done. If the relationship returns to the starting node after following the wait-for edges in the cycle, a deadlock can be concluded.
- Because of step (6) in the algorithm, the transaction are totally ordered. This reduces the number of messages transfers and decreases the deadlock detection overhead. This also ensure that there will be exactly one transaction in each cycle detects the deadlock.

Q.15. Explain edge chasing algorithm.

Ans. Edge-chasing algorithm :

- In an edge-chasing algorithm, the presence of a cycle in a distributed graph structure is verified by propagating special messages called probes, along the edges of the graph.
- These probe messages are different than the request and reply messages.
- The formation of cycle can be deleted by a site if it receives the matching probe sent by it previously.
- Whenever a process that is executing receives a probe message, it discards this message and continues.
- Only blocked processes propagate probe messages along their outgoing edges.
- Main advantage of edge-chasing algorithms is that probes are a fixed size message which is normally very short.

Q.16. Explain Chandy-Misra-Haas's edge chasing algorithm.

Ans. Chandy-Misra-Haas's algorithm :

- Chandy-Misra-Haas's distributed deadlock detection algorithm for AND model is based on edge-chasing.
- The algorithm uses a special message called probe, which is a triplet (i, j, k) , denoting that it belongs to a deadlock detection initiated for process P_i and it is being sent by the home site of process P_j to the home site of process P_k .

- A probe message travels along the edges of the global WFG graph, and a deadlock is detected when a probe message returns to the process that initiated it.
- A process P_j is said to be dependent on another process P_k if there exists a sequence of processes $P_j, P_{i1}, P_{i2}, \dots, P_{im}, P_k$, such that each process except P_k in the sequence is blocked and each process, except the P_j , holds a resource for which the previous process in the sequence is waiting.
- Process P_j is said to be locally dependent upon process P_k if P_j is dependent upon P_k and both the processes are on the same site.

Data structures :

- Each process P_i maintains a boolean array, $dependent_i$, where $dependent_i(j)$ is true only if P_i knows that P_j is dependent on it.
- Initially, $dependent_i(j)$ is false for all i and j .

Algorithm :

The following algorithm determines if a blocked process is deadlocked:

- If P_i is locally dependent on itself then declare a deadlock else for all P_j and P_k such that
 - P_i is locally dependent upon P_j ,
 - P_j is waiting on P_k ,
 - P_j and P_k are on different sites, send a probe (i, j, k) to the home site of P_k .
- On the receipt of a probe (i, j, k) , the site takes the following actions:

if

 - P_k is blocked,
 - $dependent_k(i)$ is false,
 - P_k has not replied to all requests P_j ,

then

begin

$dependent_k(i) = \text{true};$

if $k=i$

then declare that P_i is deadlocked

else for all P_m and P_n such that

(a') P_k is locally dependent upon P_m ,

(b') P_m is waiting on P_n ,

(c') P_m and P_n are on different sites, send a probe (i, m, n)
to the home site of P_n

end.

- A probe message is continuously circulated along the edges of the global WFG graph and a deadlock is detected when a probe message returns to its initiating process.
- As an example, consider the system shown in Fig. If process P_1 initiates deadlock detection, it sends probe (1, 3, 4) to the controller C_2 at site S_2 .
- Since P_6 is waiting for P_8 and P_7 is waiting for P_{10} , C_2 sends probes (1, 6, 8) and (1, 7, 10) to C_3 , which in turn sends probe (1, 9, 1) to C_1 .
- On receipt of probe (1, 9, 1), C_1 declares that P_1 is deadlocked.

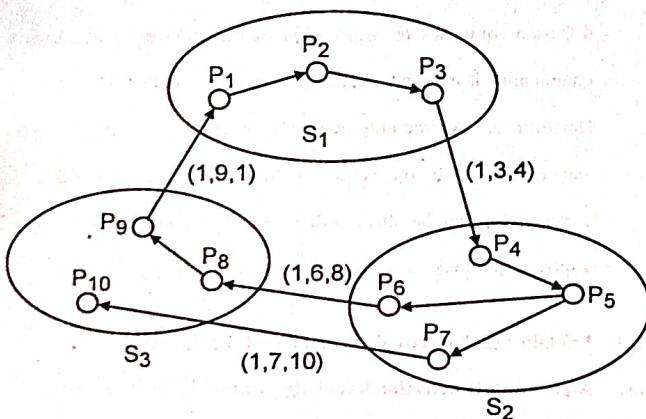


Fig. Example of Chandy-Misra-Haas's algorithm

- In the figure, if process P_1 initiates deadlock detection, it sends probe (1, 3, 4) to S_2 . Since P_6 is waiting for P_8 and P_7 is waiting for P_{10} , S_2 sends probes (1, 6, 8) and (1, 7, 10) to S_3 which in turn sends probe (1, 9, 1) to S_1 on receiving probe (1, 9, 1) S_1 declares that P_1 is deadlocked.

Performance analysis :

- One probe message (per deadlock detection initiation) is sent on every edge of the WFG which has two sites.
- Thus, the algorithm exchanges at most $m(n-1)/2$ messages to detect a deadlock that involves m processes and that spans over n sites.
- The size of messages is fixed and is very small (only 3 integer words).
- Delay in detecting a deadlock is $O(n)$.

Q.17. Explain Mitchell-Merritt edge chasing algorithm.

Ans. The Mitchell-Merritt algorithm :

- In the deadlock detection algorithm of Mitchell and Merritt each node of the TWF graph has two labels: private and public. The private label of each node is unique to that node and initially both labels at a node have the same value.
- The algorithm detects a deadlock by propagating the public label of nodes in the backward direction in the TWF graph. When a transaction is blocked the public and private label of its node in the TWF graph are changed to a value greater than their previous values and greater than the public label of the blocking transaction.
- A blocked transaction periodically reads the public label of the blocking transaction and replaces its own public label with it, provided the blocking transaction's public label is larger than its own.
- A deadlock is detected when a transaction receives its own public label. In essence, the largest public label propagates in the backward direction in a deadlock cycle.
- Deadlock resolution is simple in this algorithm because only one process detects a deadlock and that process can resolve the deadlock by simply aborting itself.

Q.18. Write about a diffusion computation based algorithm.

Ans. Diffusion computation based algorithm :

- In diffusion computation based distributed deadlock detection algorithms, deadlock detection computation is diffused through the WFG of the system.

- A process determines if it is deadlocked by initiating a diffusion computation. The messages used in diffusion computation take the form of a query (i, j, k) and a reply (i, j, k) denoting that they belong to diffusion computation initiated by a process P_i and are being sent from process P_j to process P_k .
 - A process can be in two states : active or blocked. In the active state, a process is executing and in the blocked state, a process is waiting to acquire resource.
 - A blocked process initiates deadlock detection by sending query message to all the process from whom it is waiting to receive a message (these process are called the dependent set of the process).
 - If an active process receives a query or reply message, it discards it. When a blocked process P_k receives a query (i, j, k) message, it takes the following actions :
 - If this is the first query message received by P_k for the deadlock detection initiated by P_i (called the engaging query), then it propagates the query to all the processes in its dependent set and sets a local variable $num_k(i)$ to the number of query messages sent.
 - If this is not an engaging query, then P_k returns a reply message to it immediately, provided P_k has been continuously blocked since it received corresponding engaging query. Otherwise, it discards the query.
 - A local boolean variable $wait_k(i)$ at process P_k denotes the facts that it has been continuously blocked since it received the last engaging query from process P_i . When a blocked process P_k receives a reply (i, j, k) message, it decrements $num_k(i)$ provided $wait_k(i)$ holds.
 - A process sends a reply message in response to an engaging query only after it has received a reply to every query message it sent out for this engaging query; i.e., $num_k(i) = 0$ at it.
 - An initiator detects a deadlock when it receives reply messages to all the query messages it had send out.
- The algorithm : Chandy's diffusion computation based deadlock detection algorithm in pseudocode for the OR request model.
- Initiate a diffusion computation for a blocked process P_i :
- send query (i, i, j) to all processes P_j in the

dependent set DS_i of P_i ;

$num_i(i) = |DS_i|$; $wait_i(i) := true$;

When a process P_k receives a query (i, j, k) :

if this is the engaging query for process P_k

then send query (i, k, m) to all P_m in its dependent set DS_k ;

$num_k(i) = |DS_k|$; $wait_k(i) := true$;

else if $wait_k(i)$ then send a reply (i, k, j) to P_j

When a process P_k receives a reply (i, j, k) :

if $wait_k(i)$

then begin

$num_k(i) = num_k(i) - 1$;

if $num_k(i) = 0$

then if $i = k$ then declare a deadlock

else send reply (i, k, m) to the process P_m ,

which sent the engaging query.

- In the above description of the algorithm, we assumed that only one diffusion computation is initiated for a process. In practice, several diffusion computations may be initiated for a process. A diffusion computation is initiated every time the process is blocked.
- However, at any time only one diffusion computation is current for any process. All others are outdated. The current diffusion computation can be distinguished from outdated ones by using sequence numbers.

Q.19. Explain a global state detection based algorithm.

Ans. A global state detection based algorithm :

- There are three deadlock detection algorithms to detect generalized distributed deadlocks using global state detection approach.
- The algorithm by Bracha and Toueg consists of two phases. In the first phase, the algorithm records a snapshot of a distributed WFG and in the second phase, the algorithm simulates the granting of requests to check for generalized deadlocks.
- The second phase is nested within the first phase. Therefore, the first phase terminates after the second phase has terminated.

- The algorithm by Wang also consists of two phase. In the first phase the algorithm records a snapshot of the distributed WFG. In the second phase, the static WFG recorded in the first phase is reduced to detect any deadlocks. Both the phases occur serially.
- The Kshemkalyani-Singhal algorithm has a single phase, which consists of a fan-out sweep of messages outwards from an initiator process. A sweep of a WFG is a traversal of the WFG in which all message are sent in the direction of the WFG edges (an outward sweep) or all messages are sent against the direction of the WFG edges (an inward sweep).
- Both the outward and the inward sweeps are done concurrently in the algorithm. In the outward sweep, the algorithm records a snapshot of a distributed WFG. In the inward sweep, the recorded distributed WFG is reduced to determine whether the initiator is deadlocked.
- This algorithm deals with complications introduced because the two sweep can overlap in time at any process, i.e. the reduction of the WFG at a process can begin before all the WFG edges incident at that process have been recorded.

System model :

- The system has n nodes, with every node connected to every other node by a logical channel. An event in a computation can be an internal event, a message send event, or a message receive event. Events are assigned timestamps as per Lamport's clock scheme. The timestamp of an event that occurs at time t on node i is denoted by t_i .
- The computation messages can either be REQUEST, REPLY or CANCEL message. A node i sends q_i REQUESTs to q_i other nodes when it blocks (goes from an active to a blocked state) on a p_i -out-of- q_i request. When node i blocks on node j node j becomes a successor of node i and node i becomes a predecessor of node j in the WFG. A REPLY message denotes the granting of a request. A node i unblocks when p_i out of its q_i request are granted. When the node unblocks, it sends CANCEL messages to withdraw the remaining $q_i - p_i$ requests it had sent.
- The sending and receiving of REQUEST, REPLY and CANCEL message are computation events. The sending and receiving of deadlock detection algorithm messages are algorithm events.

A node i has the following local variables to record its state :

$wait_i$: Boolean ($=$ false);

/ records the current status. */*

t_i : integer ($=$ 0); */* current time. */*

$in(i)$: set of nodes whose request are outstanding at i

$out(i)$: set of nodes on which i is waiting

p_i : integer ($=$ 0); */* the number of replies required for unblocking. */*

w_i : real ($=$ 1.0);

/ weight to detect termination of deadlock detection algorithm. */*

Request send(i) :

/ Executed by node i when it blocks on a p_i -out-of- q_i request */*

For every node j on which i is blocked do

$out(i) \leftarrow out(i) \cup \{j\};$

send REQUEST(i) to j ;

set p_i to the number of replies needed;

$wait_i \leftarrow$ true;

Request receive (j) :

/ Executed by node i when it receives a request made by j */*

$in(i) \leftarrow in(i) \cup \{j\}$

Reply send(j) :

/ Executed by node i when it replies to a request by j */*

$in(i) \leftarrow in(i) - \{j\};$

send REPLY(i) to j .

Reply receive (j) :

/ Executed by node i when it receives a reply from j to its request */*

if valid reply for the current request

then begin

$out(i) \leftarrow out(i) - \{j\};$

$p_i \leftarrow p_i - 1;$

$p_i = 0 \rightarrow$

$\{wait_i \leftarrow$ false;

$\forall k \in out(i),$ send CANCEL(i) to k ;

$out(i) \leftarrow \emptyset \}$

end

Cancel receive (j) :

*/*Executed by node i when it receives a cancel from j.*/*

if $j \in \text{in}(i)$ then $\text{in}(i) \leftarrow \text{in}(i) - \{j\}$

- When a node *init* blocks on a P-out-of-Q request, it initiates the deadlock detection algorithm. The algorithm records part of the WFG that is reachable from *init* (henceforth, referred to as *init*'s WFG) in a distributed snapshot; such a distributed snapshot includes only those dependency edges and nodes that from *init*'s WFG. When multiple nodes block concurrently, they may each initiate the deadlock detection algorithm concurrently.
- Each invocation of the deadlock detection algorithm is treated independently and is identified by the initiator's identity and initiator's timestamp when it is blocked.
- Every node maintains a local snapshot for the latest deadlock detection algorithm initiated by every other node.

HIERARCHICAL DEADLOCK DETECTION ALGORITHMS

Q.20. Write short note on Menasce-Muntz hierarchical deadlock detection algorithm.

OR Explain Ho-Ramamoorthy algorithm.

OR Explain hierarchical deadlock detection algorithm.

Ans. Hierarchical algorithm :

- In hierarchical algorithms sites are (logically) arranged in hierarchical fashion, and a site is responsible for detecting deadlocks involving only its children sites.
- To optimize performance, these algorithms take advantage of access patterns localized to a cluster of sites to optimize performance.

Menasce-Muntz algorithm :

- In the hierarchical deadlock detection algorithm of Menasce and Muntz, all the resource controllers are arranged in tree fashion. A controller manages a resource or is responsible for, deadlock detection.
- The controllers at the bottommost level, called leaf controllers, manage resources; the others, called nonleaf controllers, are responsible for deadlock detection.
- A leaf controller maintains the part of the global TWF graph that is concerned with the allocation of the resources at that leaf controller.

- A nonleaf controller maintains the TWF graph spanning its children controllers and is responsible for detecting only deadlocks involving its own leaf controllers.
- Whenever a change occurs in a controller's TWF graph due to a resource allocation, wait, or release, the change is propagated to its parent controller.
- The parent controller makes the changes in its TWF graph, searches for cycles, and propagates the changes upward if necessary.
- A nonleaf controller can be kept up to date about the TWF graphs of its children continuously (that is, whenever a change occurs) or periodically.

Ho-Ramamoorthy algorithm :

- In the hierarchical algorithm of Ho and Ramamoorthy, sites are grouped into several disjoint clusters.
- Periodically, a site is chosen as the central control site, which dynamically chooses a control site for each cluster.
- The central site requests every control site for its intercluster transaction status information and wait-for relations.
- As a result, a control site collects status tables from all the sites in its cluster and applies the one-phase deadlock detection algorithm to detect all deadlocks involving only intracluster transactions.
- Then it sends intercluster transaction status information and wait-for relations (derived from the information thus collected) to the central site.
- The central site splices the intercluster information thus received, constructs a system state graph, and searches it for cycles.
- Thus, a control site detects all deadlocks located in its cluster, and the central site detects all intercluster deadlocks.

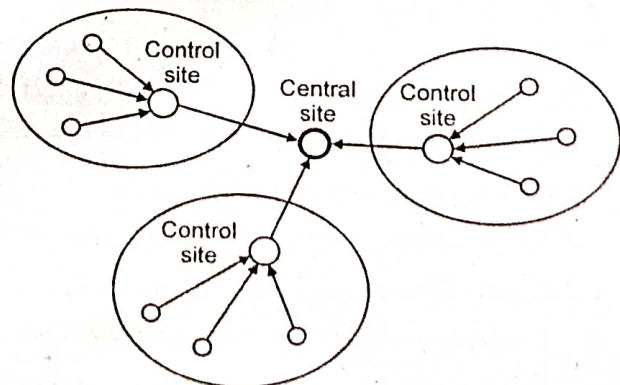


Fig. Hierarchical organization in the Ho-Ramamoorthy algorithm

AGREEMENT PROTOCOLS

Q.21. Write short note on agreement protocol.

Ans. Agreement protocol :

- In distributed systems, where sites (or processors) often compete as well as cooperate to achieve a common goal, it is often required that sites reach mutual agreement.
- For example, in distributed database systems, data managers at sites must agree on whether to commit or to abort a transaction.
- Reaching an agreement typically requires that site have knowledge about the values of other sites.
- For example, in distributed commit, a site should know the outcome of local commit at each site.
- When the system is free from failure, an agreement can easily be reached among the processor (or sites).
- For example, processor can reach an agreement by communicating their values to each other and then by taking an majority vote or a minimum, maximum, mean, etc. of those value.
- However, when the system is prone to failure, this method does not work. This is because faulty processor can send conflicting values to other processor preventing them from reaching an agreement.
- In the presence of fault, processor must exchange their value with other processors and the relay values receive from other processor several times to isolate the effect of faulty processors.
- A processor refines its value as it learns of the values of other processors. This entire process of reaching an agreement is called agreement protocol.
- The formal setting for a distributed agreement protocol is the following:
 - (i) There are M processors $P = p_1, \dots, p_m$ that are trying to reach agreement. A subset F of the processors are faulty, and remaining processors are nonfaulty. Each processor $p_i \in P$ stores a value V_i . During the agreement protocol, the processors calculate an agreement value A_i .
 - (ii) After the protocol ends, the following two conditions should hold :
 - (a) For every pair p_i and p_j of nonfaulty processors, $A_i = A_j$. This value is the agreement value.

- (b) The agreement value is a function of the initial values $\{V_i\}$ of the nonfaulty processors ($P - F$).

SYSTEM MODEL

Q.22. Explain system model with respect to agreement protocols.

OR Write a short note on :

- (1) Synchronous vs. asynchronous computations
- (2) Authenticated vs. non-authenticated messages

Ans.

- Agreement problems have been studied under following system model :
 - (i) There are n processors in the system and at most m of the processors can be faulty.
 - (ii) The processors can directly communicate with other processors by message passing. Thus, the system is logically fully connected.
 - (iii) A receiver processor always knows the identity of the sender processor of the message.
 - (iv) The communication medium is reliable (i.e. it delivers all messages without introducing any errors) and only processors are prone to failures.
- For simplicity, we assume that agreement is to be reached between only two values, 0 and 1. Result can easily be extended to multivalued agreement.
- System model includes the following parameters :
 - (1) Synchronous vs. asynchronous computations :
 - In synchronous computation, processes in the system run in lock step manner, where in each step, a process receives messages (sent to it in the previous step), performs a computation, and sends message to other processes (received in next step).
 - A step of a synchronous computation is also referred to as a round.
 - In synchronous computation, a process knows all the messages it expects to receive in a round.
 - A message delay or a slow process can slow down the entire system or computation.
 - In asynchronous computation, the computation at processes does not proceed in lock steps.

- A process can send and receive messages and perform computation at any time.
- (2) **Model of processor failures :**
A processor can fail in three modes:
 - (i) **Crash fault :**
In crash fault, a processor stops functioning and never resumes operation.
 - (ii) **Omission fault :**
In omission fault, a processor "omits" to send messages to some processors. (These are the messages that the processor should have send according to protocol or algorithm it is executing).
For example, a processor is supposed to broadcast a message to all other processors, but it sends the message to only a few processors.
 - (iii) **Malicious fault (Byzantine faults) :**
In malicious fault, a processor behaves randomly and arbitrarily.
For example, a processor may send fictitious messages to other processors to confuse them.
Malicious faults are very broad in nature and thus most conceivable faults can be treated as malicious faults.
Malicious fault is also referred to as byzantine faults.
- (3) **Authenticated vs. non-authenticated messages :**
 - In authenticated message system, a (faulty) processor cannot forge a message or change the contents of a received message (before it relay the message to other processors).
 - A processor can verify the authenticity of a received message.
 - An authenticated message is also called a signed message.
 - In non-authenticated message system, a (faulty) processor can forge a message and claim to have received it from another processor or change the contents of a received message before it relays the message to other processors.
 - A processor has no way of verifying the authenticity of a received message.
 - A non-authenticated message is also called an oral message.
 - It is easier to reach agreement in an authenticated message system because fault processors are capable of doing less damage.
- (4) **Performance aspects :**
The performance (or the computational complexity) of agreement protocols is generally determined by the following three metrics:

(i) **Time :**

Time refers to the time taken to reach an agreement under protocol. The time is usually expressed as the number of rounds needed to reach an agreement.

(ii) **Message traffic :**

Message traffic is measured by the number of messages exchanged to reach an agreement. Sometimes, the message traffic is also measured by the total number of bits exchange to reach an agreement.

(iii) **Storage overhead :**

Storage overhead measures the amount of information that needs to be stored at processors during the execution of a protocol.

CLASSIFICATION OF AGREEMENT PROBLEMS

Q.23. Explain the Byzantine agreement problem.

OR Explain three agreement problems in distributed systems.

Ans. Agreement problems :

- There are three well known agreement problems in distributed systems: the Byzantine agreement problem, the consensus problem and the interactive consistency problem.
- In the Byzantine agreement problem, a single value, which is to be agreed on, is initiated by an arbitrary processor and all nonfaulty processors have to agree on that value.
- In the consensus problem, every processor has its own initial value and all nonfaulty processors must agree on a single common value.
- In the interactive consistency problem, every processor has its own initial value and all nonfaulty processors must agree on a set of common values.
- In all three problems, all nonfaulty processors must reach a common agreement.
- In the Byzantine agreement and the consensus problems, the agreement is about a single value.
- Whereas in interactive consistency problem, the agreement is about a set of common value.
- In Byzantine agreement problem, only one processor initializes the initial value.

- Whereas in the consensus and the interactive consistency problems, every processor have its own initial value.

The three agreement problems :

Problem	Byzantine	Consensus	Interactive Consistency
Who initiates the value	One processor	All processors	All processors
Final agreement	Single value	Single value	A vector of value

(1) The Byzantine agreement problem :

- An arbitrarily chosen processor, called the source processor, broadcasts its initial value to all other processors.
- The Byzantine agreement problem was first defined and solved (under processor failure) by Lamport.
- In this problem, an arbitrarily chosen processor (called the source processor) broadcasts its initial value to all other processors.
- A protocol for Byzantine agreement should guarantee that all nonfaulty processor agree on the same value and if the source processors should be the initial value of the source.
- It is obvious that all the processors must exchange the values through messages to reach a consensus.
- Processors send their values to other processor and relay received message value to other processors.
- During the execution of protocol, faulty processors may confuse other processors by sending them conflicting values or by relaying to them fictitious values.
- The Byzantine agreement problem is also referred to as Byzantine generals problem because the problem resembles a situation where a team of generals in an army is trying to reach agreement on attack plan.
- The general are located at geographically distant positions and communicate only through messengers.
- Some of the generals are traitors (equivalent to faulty processors) and try to prevent loyal generals from reaching an agreement by deliberately transmitting erroneous information.

- A solution to the Byzantine agreement problem should meet the following two objectives:

- Agreement :** All nonfaulty processors agree on the same value
- Validity :** If the source processor is nonfaulty, then the common agreed upon value by all nonfaulty processors should be initial value of the source.

- Two points should be noted :

- If the source processor is faulty, then all nonfaulty processors agree on any common value.
- It is irrelevant what value faulty processors agree on or whether they agree on a value at all.

(2) The consensus problem :

- Every processor broadcasts its initial value to all other processors. Initial values of the processors may be different.
- A protocol for reaching consensus should meet the following conditions:
 - Agreement :** All nonfaulty processors agree on the same single value.
 - Validity :** If the initial value of every nonfaulty processor is v , then agreed upon common value by all nonfaulty processor must be v .
- Note that if the initial values of nonfaulty processors are different, then all nonfaulty processors can agree on any common value.

(3) The interactive consistency problem :

- Every processor broadcasts its initial value to all other processors. The initial values of the processors may be different.
- A protocol for the interactive consistency problem should meet the following conditions:
 - Agreement :** All nonfaulty processors agree on the same vector, $(v_1, v_2, \dots, v_n)$.
 - Validity :** If the i^{th} processor is nonfaulty and its initial value is v_i , then the i^{th} value to be agreed on by all nonfaulty processors must be v_i .
- Note that if the j^{th} processor is faulty, then all nonfaulty processors can agree on any common value for v_j .

Q.24. Explain in brief about the relations among the agreement problems.

Ans. Relations among the agreement problems :

- All three agreement problems are closely related. For example, the Byzantine agreement problem is a special case of the interactive consistency problem, in which the initial value of only one processor is of interest.
- Conversely, if each of the n processors runs a copy of the Byzantine agreement protocol, the interactive consistency problem is solved.
- Likewise, the consensus problem can be solved using the solution of the interactive consistency problem. This is because all nonfaulty processors can compute the value that is to be agreed upon by taking the majority value of the common vector that is computed by an interactive consistency protocol, or by choosing a default value if a majority does not exist.
- Thus solutions to the interactive consistency and consensus problems can be derived from the solutions to the Byzantine agreement problem. In other words, the Byzantine agreement problem is primitive to the other two agreement problems.
- However, it should by no means be concluded that the Byzantine agreement problem is weaker than the interactive consistency problems or that the interactive consistency problem is weaker than the consensus problem. In fact, there is no linear ordering of this sort among these agreement problems. For example, the Byzantine agreement problem can be solved using a solution to the consensus problem in the following manner.
 - (i) The source processor send its value to all other processors, including itself.
 - (ii) All the processors, including the source, run an algorithm for the consensus problem using the values received in the first step as their initial values.
- The above two steps solve the Byzantine agreement problem because (i) if the source processor is nonfaulty, then all the processors will receive the same value in the first step and all nonfaulty processors will agree on the value as a result of the consensus algorithm in the second step, and (ii) if the source processor is faulty, then the other processors may not receive the same value in the first step; However, all nonfaulty processors will agree on the same value in

the second step as a result of the consensus algorithm. Thus, the Byzantine agreement is reached in both cases. However, then $n-1$ extra messages are sent at the first step.

SOLUTIONS TO THE BYZANTINE PROBLEM

Q.25. Explain the concept of upper bound on the number of faulty processors.

Ans. The upper bound on the number of faulty processors :

- In order to reach agreement on a common value, nonfaulty processors need to be free from the influence of faulty processors.
- In faulty processors dominate in number, they can prevent nonfaulty processors from reaching a consensus.
- Thus the number of faulty processors should not exceed a certain limit if a consensus is to be reached.
- Pease showed, that in faulty connected network, it is impossible to reach a consensus if the number of faulty processors, m , exceeds $[(n-1)/3]$.
- Lamport was the first to give protocol to reach Byzantine agreement that requires $m+1$ rounds of message exchange (m is the maximum number of faulty processors).
- Fischer showed that $m+1$ is the lower bound on the number of rounds of message exchange to reach a Byzantine agreement in a faulty connected network where only processors can fail.
- However, if authenticated messages are used, this bound is relaxed and a consensus can be reached for any number of faulty processors.

Q.26. Explain the concept of an impossible result as a solution of Byzantine problem.

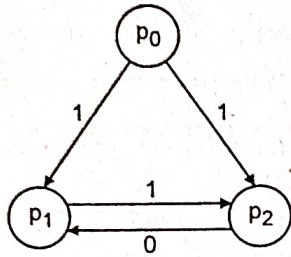
Ans. An impossible result :

- We now show that a Byzantine agreement cannot be reached among three processors, where one processor is faulty.
- Consider a system with three processors, p_0 , p_1 and p_2 . For simplicity, we assume that there are only two values, 0 and 1, on which processors agree and processor p_0 initiates the initial value.

Case I : p_0 is not faulty :

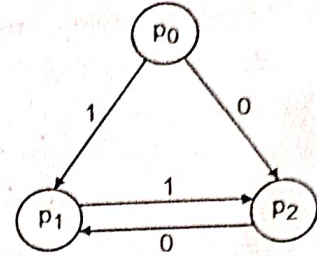
- Assume p_2 is faulty. Suppose that p_0 broadcast an initial value of 1 to both p_1 and p_2 .

- Processor p_2 acts maliciously and communicates a value of 0 to processor p_1 . Thus p_1 receives conflicting values from p_0 and p_2 .
- This scenario is shown in Fig. (a). A faulty processor is depicted by an oval and a nonfaulty processor is denoted by a circle.
- However, since p_0 is nonfaulty, processor p_1 must accept 1 as the agreed upon value if condition 2 is to be satisfied.

Fig.(a) Processor p_0 is Non-Faulty

Case II : p_0 is faulty :

- Suppose that processor p_0 sends an initial value of 1 to p_1 and 0 to p_2 . Processor p_2 will communicate the value 0 to p_1 . It is shown in Fig. (b).
- As far as p_1 is concerned, this case will look identical to case I.
- So any agreement protocol which works for three processor cannot distinguish between the two cases and must force p_1 to accept 1 as the agreed upon value whenever p_1 is faced with such situations (to satisfy condition 2).
- However in case II, this will work only if p_2 is also made to accept 1 as the agreed upon value.
- Using similar argument, we can show that if p_2 receive an initial value of 0 from p_0 , then it must take 0 as the agreed upon the value, even if p_1 communicates a value of 1.
- However, if this is followed in Case II, p_1 will agree on a value of 1 and p_2 will agree on a value of 0, which will violate condition 1 of the solution.
- Therefore, no solution exists for the Byzantine agreement problem for three processors, which can work under single processor failure.

Fig.(b) p_0 is faulty

Q.27. Explain Lamport-Shostak-Pease algorithm.

Ans. Lamport-Shostak-Pease algorithm :

- Lamport's algorithm, referred to as the Oral Message algorithm OM(m), $m > 0$, solves the Byzantine agreement problem for $3m+1$ or more processors in the presence of at most m faulty processors. Let n denote the total number of processors (clearly, $n \geq 3m+1$).

Algorithm OM(0) :

- (1) The source processor sends its value to every processor.
- (2) Each processor uses the value it receives from the source. (If it receives no value, then it uses a default value of 0)

Algorithm OM(m), $m > 0$:

- (1) The source processor sends its value to every processor.
 - (2) For each i , let v_i be the value processor i receives from the source. (If it receives no value, then it uses a default value of 0.). Processor i acts as the new source and initiates Algorithm OM($m-1$) wherein it sends the value v_i to each of the $n-2$ other processors.
 - (3) For each i and each j ($j \neq i$), let v_{ij} be the value processor i received from processor j in Step 2 using Algorithm OM($m-1$). Processor i uses the value majority ($v_{i1}, v_{i2}, \dots, v_{in-1}$).
- This algorithm is evidently quite complex. The processor are successively divided into smaller and smaller groups and the Byzantine agreement is recursively achieved within each group of processor (Step 2 of "Algorithm OM($m-1$)").
 - Step 3 is executed during the folding phase of the recursion, where a majority function is applied to select the majority value out of the values received in a round of message exchange (Step 2).
 - The function majority ($v_{i1}, v_{i2}, \dots, v_{in-1}$) compute a majority value of the values $v_{i1}, v_{i2}, \dots, v_{in-1}$ if it exists (otherwise, it returns the default value 0).

- The execution of algorithm OM(m) invokes $n-1$ separate executions of the algorithm OM(m-1), each of which invokes $n-2$ executions of the algorithm OM(m-2), and so on.
- Therefore, there are $(n-1)(n-2)(n-3)\dots(n-k)$ separate execution of algorithm, OM(m-k), $k = 1, 2, 3, \dots, m+1$.
- The message complexity of the algorithm is $O(n^m)$.

Example 1 :

- Consider a system with four processors, p, p_1, p_2 , and p_3 . For simplicity, we assume that there are only two values 0 and 1; furthermore, we assure that processor p_0 initiates the initial value and that processor p_2 is faulty.
- To initiate the agreement, processor p_0 executes algorithm OM(1) wherein it sends its value 1 to all other processors (Fig.(a)). At step 2 of the algorithm OM(1), after having received the value 1 from the source processor p_0 , processors p_1, p_2 , and p_3 execute the algorithm OM(0).
- These executions are shown in Fig.(b). Processors p_1 and p_3 are nonfaulty and send value 1 to processors $\{p_2, p_3\}$ and $\{p_1, p_2\}$, respectively. Faulty processor p_2 sends value 1 to p_1 and a value 0 to p_3 (see Fig. (b)).
- After having received all the messages, processors p_1, p_2 and p_3 execute Step 3 of the algorithm OM(1) to decide on the majority value. Processor p_1 has received values (1, 1, 1), whose majority value is 1, processor p_2 has received values (1, 1, 1), whose majority value is 1, and processor p_3 has received values (1, 1, 0), whose majority value is 1. Thus, both conditions of the Byzantine agreement are satisfied.

Example 2 :

- Fig.(c) shows a situation where processor p_0 is faulty and sends conflicting values to the other three processors. These three processors, under Step 1 of OM (0), send the received values to the other two processors.
- After having received all the messages, processors p_1, p_2 and p_3

execute Step 3 of the algorithm OM(1) to decide on the majority value. Note that all three processor have received values (1, 0, 1), whose majority value is 1. Thus, all three nonfaulty processors agree on the same value and the required conditions of the Byzantine agreement are satisfied.

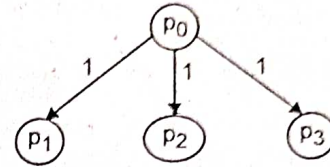


Fig. (a) Processor p_0 executes the algorithm OM(1)

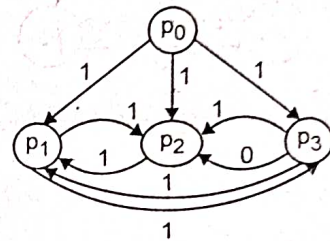


Fig. (b) Processors p_1, p_2 and p_3 execute the algorithm OM(0)

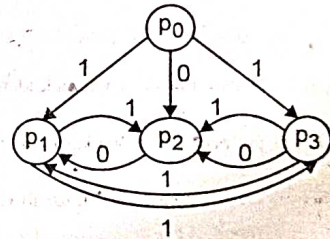


Fig. (c) Processors p_0, p_1, p_2 and p_3 execute OM(1) and OM(0)

Q.28. Explain Dolev's algorithm.

Ans. Dolev's algorithm :

- Dolev have given a polynomial algorithm for reaching Byzantine agreement. The algorithm is polynomial in message complexity.
- However, the algorithm requires up to $2m+3$ rounds to reach an agreement. Thus, there is a trade off between message complexity and time delay (rounds).

Data structure :

- The algorithm uses two thresholds: LOW and HIGH, where $LOW := m+1$ and $HIGH := 2m+1$.
- The basic idea is that any subset of processors of size LOW will have at least one nonfaulty processor.

- Any subset of processors of size HIGH includes a majority of processors, that is, $m+1$, that are nonfaulty.
 - The algorithm uses two types of messages: a "*" message and a message consisting of the name of a processor.
 - The "*" denotes the fact that the sender of the message is sending a value of 1 and the name in a message denotes the fact that the sender of the message received a "*" from the named processor.
 - Let W_k^i be the set of processors that have sent message x to processor i . (Note that x is either a "*" or a processor name.)
 - Each process maintains $n+1$ numbers of W sets.
 - We refer to W_k^i as the set of witnesses of message x for processor i .
 - A processor j is a direct supporter for a processor k if j directly receives "*" from k .
 - When a nonfaulty processor j directly receives "*" from processor k , it sends message 'k' to all other processors.
 - When processor i receives the message "k" from processor j , it adds j into W_k^i because j is a witness to message "k". Process j is an indirect supporter for processor k if $|W_k^j| \geq \text{LOW}$;
 - A processor j confirms processor k if $|W_k^j| \geq \text{HIGH}$;
- The algorithm :**
- In the first round the source processor sends a "*" message to all processors (including itself) if its value is 1. If its value is 0, it sends nothing in the first round. If the processors finally agree on "*", then the agreed upon value is 1. Otherwise, the agreed upon value is 0.

- In subsequent rounds; a processor sends its message to all processors, receives messages from other processors, and then decides what messages to send in the next round.

Initiation operation :

A processor initiates under the following conditions :

- It initiates in the second round if it receives a "*" from the source in round 1.
- It initiates in the $K+1$ st round if at the end of K th round the cardinality of the set of the confirmed processors (not including the source) at least $\text{LOW} + \max [0, K/2] - 2$ (referred to as the condition of initiation).

The following four rules describe the operation of the Dolev's algorithm :

- In the first round, the source broadcasts its value to all other processors.
- In a round $k > 1$, a processor broadcasts the names of all processes for which it is either a direct or indirect supporter or which it has not previously broadcast. If the condition of initiation was true at the end of the previous round, it also broadcasts the "*" message unless it has previously done so.
- If a processor confirms HIGH number of processors, it commits to a value of 1.
- After round $2m+3$, if the value 1 is committed, the processors agree on 1; otherwise, they agree on 0.

POINTS TO REMEMBER :

- Out of the three approaches to handle deadlocks, deadlock detection is the most promising for distributed system.
- In centralised deadlock detection algorithms, the content site has the responsibility of constructing global state graph and searching it for cycles.
- In distributed deadlock detection algorithms, every site maintains a portion of the global state graph and every site participates in the detection of a global cycle.
- Hierarchical deadlock detection algorithms fall between centralised and distributed deadlock detection algorithm and exploit access patterns local to a cluster of sites to detect deadlock efficiently.
- In distributed systems, it is often required that sites reach a mutual agreement.
- The agreement problem is non solvable in an asynchronous systems even for a single processor failure.
- In an asynchronous computation, computation at processors does not proceed in lock steps.